Implement phase 07 API

• Project: Sonrisa

• Storage: active

• Session ID: 01a19212-00fd-7510-9e71-677fd83d8823

• Started: 2026-10-31T13:06:56.243Z

• Updated: 2026-10-31T13:31:26.689Z

• Model: gpt-6.1-sol

This PDF is a classified, derived reading view. Raw JSONL remains the canonical lossless representation when included.

Automatic runtime context – ENVIRONMENT – 2026-10-31T13:07:22.689Z

<environment_context>
<cwd>D:\Temp\Sonrisa</cwd>
<shell>powershell</shell>
<current_date>2026-10-31</current_date>
<timezone>Europe/Budapest</timezone>
<filesystem><workspace_roots><root>D:\Temp\Sonrisa</root></
workspace_roots><permission_profile type="managed"><file_system type="restricted"><entry
access="read"><special>:root</special></entry><entry access="write"><path>D:\Temp\Sonrisa</
path></entry><entry access="write"><special>:slash_tmp</special></entry><entry
access="write"><special>:tmpdir</special></entry><entry access="read"><path>D:
\Temp\Sonrisa\.git</path></entry><entry access="read"><path>D:\Temp\Sonrisa\.agents</path></
entry><entry access="read"><path>D:\Temp\Sonrisa\.codex</path></entry><entry
access="read"><path>D:\Temp\Sonrisa\.aws</path></entry></file_system></permission_profile></
filesystem>
</environment_context>

Automatic runtime context – 2026-10-31T13:07:22.699Z

<external_codex_apps_open_page>{"page_id":null}</external_codex_apps_open_page>

User – 2026-10-31T13:07:22.710Z

Implement Phase 07 only, following `AGENTS.md`, `PROJECT_CONTEXT.md`, `PHASES.md`, and `phases/04-api.md`.\
Review the existing repository, explain your implementation plan, and ask any blocking questions before coding.\
Complete the phase's acceptance criteria, run available tests, update `IMPLEMENTATION_STATUS.md`, and stop for my explicit validation. Do not implement any subsequent phases.

Assistant – 2026-10-31T13:07:25.458Z

I'll review the repository guidance and phase definitions, then explain the Phase 07 plan and flag any blocking questions before coding.

Assistant – 2026-10-31T13:07:49.247Z

Phase 06 is recorded as human-approved, and the working tree is clean. The requested `phases/04-api.md` does not exist; the Phase 07 specification is `codex_alerts_plan/phases/07-api.md`, which I'll follow.

The repository already has cookie authentication, CSRF validation, server authorization policies, independent selection services, cascade-delete mappings, and dispatch enablement checks. My plan is to:

- Add personal catalog and selection endpoints using the authenticated session's user ID.
- Add admin user listing, enable/disable and deletion, plus global topic/channel controls.
- Keep disabled catalog items visible with their status, preserve existing preferences, and protect every mutation with CSRF validation.
- Add Development-only OpenAPI documentation, focused API/policy tests, and sample validation requests.
- Run the available checks, update `IMPLEMENTATION_STATUS.md` to `AWAITING HUMAN VALIDATION`, and stop.

Two behavior choices need your decision before coding.

Assistant – 2026-10-31T13:07:56.280Z

Review complete; the implementation plan is above. I'll use the existing `codex_alerts_plan/phases/07-api.md` because `phases/04-api.md` is absent.

Before coding, please answer the two pending questions about selecting disabled items and admin self-disable/delete. No files have changed.

User – 2026-10-31T13:08:27.170Z

<send_user_message_question_reply>
[{"questionItemId":"[\"request_user_input_async\",\"call_8d45c85bd7e6485a8daf2c0e9893afbb\",0]","question":"May users newly select a globally disabled topic/channel? Existing selections will remain stored and dispatch will stay suppressed.","answer":"Reject new selections while disabled; allow removal (Recommended)"},{"questionItemId":"[\"request_user_input_async\",\"call_8d45c85bd7e6485a8daf2c0e9893afbb\",1]","question":"Should the API prevent an admin from disabling or deleting their own account?","answer":"Prevent self-disable and self-delete (Recommended)"}]

</send_user_message_question_reply>

Assistant – 2026-10-31T13:08:31.612Z

I'll reject new selections of disabled items while preserving existing selections and allowing removal. I'll also block admin self-disable and self-delete. I'll now implement Phase 07 and its validation coverage.

Assistant – 2026-10-31T13:10:00.242Z

The endpoints are in place with session-derived ownership, CSRF protection, safe response fields, and the two agreed guards. Development-only OpenAPI generation is configured using ASP.NET Core's supported package (https://learn.microsoft.com/en-us/aspnet/core/fundamentals/openapi/aspnetcore-openapi?view=aspnetcore-10.0).

Next I'll test the authorization boundaries and verify that API enablement changes suppress dispatch. I'll also check deletion separately against PostgreSQL, since the existing EF InMemory test store cannot prove database cascades.

Assistant – 2026-10-31T13:13:45.540Z

The first test run found a fixture setup error: it initialized catalog seeds after Identity had already created the InMemory store, leaving the catalogs empty. I've corrected the initialization order and am rerunning the checks.

The isolated Docker stack also needed its generated environment file corrected. It now has a fresh Phase 07 database for validating HTTP deletion and the actual PostgreSQL cascades.

Assistant – 2026-10-31T13:15:33.305Z

The live API checks passed: the regular user could manage independent preferences, received 403 for admin access, and received 400 for a mutation without CSRF. Both admin self-action guards returned 409.

PostgreSQL validation also passed. An alert published while its topic was disabled produced no delivery; a new alert after re-enabling produced one Mailpit message. Deleting the test user returned 204 and removed its selections and delivery, while retaining both shared events and all catalog rows. The deleted user's existing session then returned 401.

Assistant – 2026-10-31T13:18:22.418Z

The build, formatting check, and Compose configuration check pass. The full test rerun is underway after correcting a cache-header assertion to check for `no-store` while allowing Identity's additional `no-cache` directive.

The API guide now includes request examples and manual acceptance steps. I've stopped the

isolated validation services and retained their data for inspection.

Assistant – 2026-10-31T13:19:51.422Z

All 156 tests pass with no failures or skips. The build has zero warnings/errors, and formatting and Compose configuration checks pass.

I'm recording the final evidence in `IMPLEMENTATION_STATUS.md` and leaving Phase 07 at `AWAITING HUMAN VALIDATION`. No subsequent phase has been implemented.

Assistant – 2026-10-31T13:20:00.788Z

Phase 07 is implemented and **AWAITING HUMAN VALIDATION**.

- Added personal/admin APIs, CSRF protection, agreed selection/self-action guards, and Development-only OpenAPI.

- Validation passed: build, all **156 tests**, formatting, Compose configuration, and whitespace checks.

- Live PostgreSQL/Mailpit checks confirmed dispatch suppression and deletion cascades.

Changed-file details and test evidence are in IMPLEMENTATION_STATUS.md [local file not included] . Follow the API validation guide [local file not included] for manual review.

Used the existing `07-api.md`; the requested `04-api.md` was absent. No blocking questions remain. No subsequent phase or commit was created.

Please explicitly approve Phase 07 or provide corrections.

Automatic runtime context – 2026-10-31T13:23:43.204Z

<external_codex_apps_open_page>{"page_id":null}</external_codex_apps_open_page>

User – 2026-10-31T13:23:43.213Z

explain to me the logic behind ManagementEndpoints class over using a classical controller setup

Assistant – 2026-10-31T13:24:02.181Z

`ManagementEndpoints` uses ASP.NET Core **Minimal APIs**. It serves the same HTTP boundary role as a controller: mapping routes, binding requests, enforcing authorization, and returning responses.

I chose it mainly for consistency with the repository. Authentication already uses `AuthEndpoints`, and `Program.cs` registers endpoint groups directly. Phase 07 follows that pattern without introducing a second routing convention.

The groups also make shared rules explicit:

- `/api/me` requires authenticated user access.
- `/api/admin` requires the admin policy.
- Every mutation uses the existing CSRF filter.
- Each group shares documentation tags and cache behavior.

Controllers could enforce all those rules equally well through attributes and filters. Minimal APIs offer no inherent security or architectural advantage here.

The tradeoff is organization. `ManagementEndpoints` now combines personal settings, catalog administration, and user administration in one class. Some handlers also access EF directly. That kept this small phase straightforward, but moving to controllers alone would not improve separation of responsibilities–the database operations would still need to move into application services.

For this repository, the choice was reasonable because it matches the existing structure. For continued growth, I'd split the HTTP boundary into personal settings and administration classes, using either Minimal APIs or controllers according to your preferred convention. Classical controllers would be an equally valid choice, especially if your team finds their structure easier to navigate and maintain.

User – 2026-10-31T13:26:47.904Z

Do this: split the HTTP boundary into personal settings and administration classes, using Minimal API's

Assistant – 2026-10-31T13:26:51.240Z

I'll split the routes into personal settings and administration classes, keep the API behavior unchanged, and rerun the checks. Phase 07 will remain awaiting your validation.

Assistant – 2026-10-31T13:27:24.327Z

`Program.cs` currently registers all management routes through `MapManagementEndpoints`. I'll replace that with `MapPersonalSettingsEndpoints` and `MapAdministrationEndpoints`, each owning its handlers. Shared catalog reads, response contracts, and HTTP helpers will stay in small supporting files. There are no blocking questions for this refactor.

Assistant – 2026-10-31T13:29:19.209Z

The split is complete: personal settings and administration now have separate Minimal API classes and registrations in `Program.cs`. The build and formatting checks pass. The existing HTTP tests are running to verify routing, authorization, CSRF, and OpenAPI behavior.

Assistant – 2026-10-31T13:30:17.664Z

A comparison confirms that all 14 route mappings are unchanged. Formatting and whitespace checks also pass; I'm waiting for the full test suite to finish before recording the final result.

Assistant – 2026-10-31T13:30:36.852Z

Split the Minimal API boundary into:

- PersonalSettingsEndpoints.cs [local file not included] — personal settings.
- AdministrationEndpoints.cs [local file not included] — administration.

`Program.cs` registers each independently. Shared helpers and contracts live in supporting files. Routes, security policies, and API behavior remain unchanged.

Build, all **156 tests**, formatting, and whitespace checks pass. Documentation and implementation status are updated.

Phase 07 remains **AWAITING HUMAN VALIDATION**. Please approve or provide feedback.

User – 2026-10-31T13:31:01.320Z

with these changes the step is approved, update status files

Assistant – 2026-10-31T13:31:04.940Z

I'll record your approval of Phase 07, including the endpoint split, and update the status references. Phase 08 will remain not started.

Assistant – 2026-10-31T13:31:26.665Z

Recorded Phase 07 as **APPROVED**, including the endpoint split, in the implementation log, README, and API guide.

Documentation-only update; whitespace check passed. Phase 08 remains **NOT STARTED**.